

CLI Based Banking Management System

Report

Group 11-Variant 3

By:

Harmit Patel (110164115)

Haotian Yang (110105670)

Guided By:

Dr. Olena Syrotkina

SCHOOL OF COMPUTER SCIENCE

COMP2120

Object-Oriented Programming Using Java

Interession/Summer 2024



University
of Windsor

The requirement of this project:

CLI-Based Banking System

Project Description: Develop a console application for managing a banking system that allows customers to open accounts, perform transactions (deposits, withdrawals, transfers), manage account information, and view account balances. The system will also provide administrative functionalities for bank staff, such as account monitoring and report generation.

Core Features:

User and Account Registration: Adding customers and creating bank accounts.

Transaction Management: Handle deposits, withdrawals, and transfers.

Account Management: Update account information and view balances.

Search Functionality: Search for accounts by customer name, account number, or balance.

Report Generation: Generate reports about bank accounts in any format of your choice (e.g. *.txt)

Technical Specifications:

Introduction to Java: Use Java's basic syntax and control structures to handle user input and display outputs.

Objects and Classes:

Create classes for accounts, transactions, and users.

Inheritance and Polymorphism:

Differentiate between different types of bank accounts (e.g., Checking, Savings) using inheritance.

Generics:

Use generics to manage lists of transactions and accounts securely.

Exception Handling:

Implement robust exception handling for errors during transaction processing and data entry.

Collections and Data Structures:

Utilize Java collections (e.g. ArrayList) to manage user and account data.

Multithreading:

Handle concurrent access to account data and transaction processing.

I/O Programming:

Read and write data (e.g. user info, transaction logs) to files.

Possible Java Classes and Interfaces:

Account.java: Represents a bank account.

Transaction.java: Represents a financial transaction.

User.java: Represents a user of the bank system.

Bank.java: Manages accounts, transactions, and users.

BankInterface.java: Handles the CLI interactions.

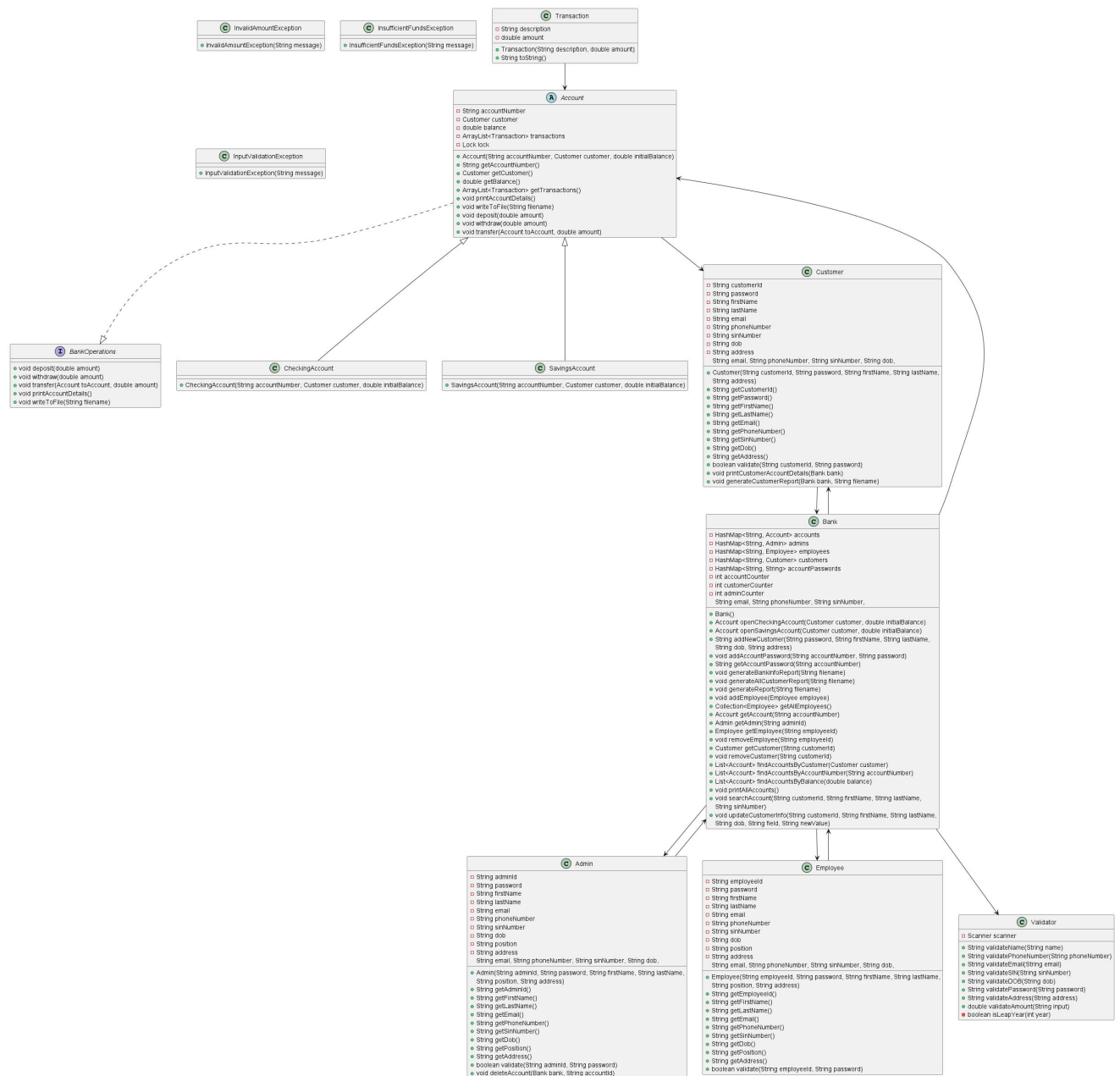
Development Steps:

Design the System: Outline data models and user interfaces.

Implement Core Functionalities: Develop the backend operations and user interface.

Integrate Advanced Features: Add multithreading and enhanced security measures.

Testing and Refinement: Conduct thorough testing and refine based on feedback.



SYSTEM DIAGRAM WITH WORKFLOW



Overview

The CLI (Command Line Interface) Banking System is a comprehensive project designed to simulate real-world banking operations. It allows users to perform various banking functions, such as account management, transactions, and report generation, all within a command-line interface. The system is developed using object-oriented programming principles, including inheritance, polymorphism, generics, threading, and exception handling. This overview provides a detailed explanation of each component of the system, its functionality, and the overall purpose of the system.

System Purpose

The CLI Banking System is designed to:

Simulate Real-World Banking Operations: The system mimics essential banking operations like account creation, deposits, withdrawals, transfers, and generating reports, providing users with a realistic experience of bank management.

Educate on Object-Oriented Programming (OOP): The system serves as an educational tool, demonstrating the application of OOP concepts like inheritance, polymorphism, and exception handling in a real-world context.

Enhance User Interaction Through CLI: It provides a user-friendly command-line interface, making it accessible and straightforward for users to interact with the system and perform banking operations.

Demonstrate Robust Error Handling: The system includes comprehensive error handling to ensure that user inputs are validated and managed effectively, preventing system crashes and maintaining data integrity.

Ensure Thread Safety in Multi-User Environment: By incorporating threading and concurrency controls, the system can handle multiple users interacting with the system simultaneously, simulating a real-world banking environment.

Detailed Breakdown of System Components

Main Class:

The entry point of the application where the program initializes the Bank object and handles user interactions through a series of command-line menus.

It presents different options for Admin, Employee, and Customer logins, guiding users to specific functionalities based on their roles.

Bank Class:

Acts as the central hub for managing all banking operations. It stores collections of accounts, admins, employees, and customers, and provides methods for performing various banking functions.

Account Management:

Methods to create, delete, and search for accounts. It also includes features for updating customer information and managing employee records.

Transaction Management: Facilitates deposits, withdrawals, and transfers between accounts. It ensures that all transactions are thread-safe, preventing data inconsistencies.

Report Generation: Allows Admins and Employees to generate detailed reports on accounts, customers, and overall bank information. These reports include transaction histories, balances, and personal details.

Account Class (Abstract):

Serves as a base class for all types of bank accounts (Checking Account and Savings Account). It encapsulates common attributes and methods shared by all account types, such as account number, customer details, balance, and transaction history.

Methods:

deposit(), withdraw(), and transfer() methods handle basic transactions, ensuring that operations are thread-safe using the ReentrantLock.

printAccountDetails() and writeToFile() methods for displaying and saving account information.

CheckingAccount and SavingsAccount Classes:

Subclasses of Account, representing specific types of bank accounts. These classes inherit from the Account class and provide specialized implementations for checking and savings accounts.

Transaction Class:

Represents individual transactions within an account. Each transaction has a description and an amount, and it is stored within an account's transaction history.

Methods:

toString() method returns a string representation of the transaction, which is used for printing or saving transaction details.

Admin Class:

Represents the administrative users of the system. Admins have the highest level of access and can manage accounts, employees, and generate reports.

Methods:

validate() method for authenticating admin credentials.

Methods for adding, updating, and deleting customer and employee records, along with generating detailed reports on all users in the system.

Employee Class:

Represents bank employees who manage customer accounts. Employees have limited access compared to Admins but can perform key banking operations like account management and report generation.

Methods:

validate() method for authenticating employee credentials.

Methods for performing customer-related operations such as opening accounts, processing transactions, and updating customer details.

Customer Class:

Represents bank customers who hold and manage their own accounts. Customers can perform personal banking operations like deposits, withdrawals, and transfers, and can generate reports for their own accounts.

Methods:

validate() method for authenticating customer credentials.

printCustomerAccountDetails() and generateCustomerReport() methods for displaying and saving the customer's account information.

Validator Class:

Provides input validation methods to ensure that all user inputs are correct before they are processed by the system.

Methods:

Various methods for validating names, phone numbers, emails, SINS, DOBs, passwords, and addresses. These methods ensure that inputs meet specific criteria and throw custom exceptions if invalid data is detected.

Custom Exceptions:

`InvalidAmountException`: Thrown when a transaction involves an invalid amount (e.g., negative or zero).

`InsufficientFundsException`: Thrown when a withdrawal or transfer operation is attempted with insufficient funds in the account.

`InputValidationException`: Thrown when user inputs fail to meet the validation criteria defined in the Validator class.

Threading and Concurrency Control:

The system uses `ReentrantLock` to manage concurrent access to account data. This ensures that multiple users can perform transactions on the same account without causing data inconsistencies.

Methods:

Critical sections of code in transaction methods (`deposit()`, `withdraw()`, `transfer()`) are protected by locks, ensuring that only one thread can modify account balances at a time.

The Additional Features

Additional Features Developed

The CLI Banking System project has been significantly enhanced beyond the basic requirements to provide a more robust, flexible, and user-friendly experience. The additional features developed include:

Dynamic Account Management:

Opening Accounts: The system allows Admins and Employees to open both checking and savings accounts dynamically. The process involves detailed input collection and validation for customer information, ensuring that each account is created with accurate and complete data. This feature not only adds flexibility to account management but also helps simulate real-world banking operations where customers can choose between different types of accounts based on their needs.

```
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 1
1. Open Checking Account
2. Open Savings Account
Enter your choice: 1
Enter first name: 123
Invalid input: Name must contain only letters. Enter again:
yht
Enter last name: yht
Enter password: y@
Invalid input: Password must be at least 5 characters long. Enter again:
y@123
Enter email: yht@gmail.com
Enter phone number: 123456789
Invalid input: Phone number must be exactly 10 digits. Enter again:
1234567890
Enter SIN number: 12
Invalid input: SIN must be exactly 9 digits. Enter again:
123456789
Enter DOB (MM/DD/YYYY):
```

Deleting Accounts: Admins and Employees have the ability to delete customer accounts, with the system checking if the account exists before performing the deletion. This ensures that only valid accounts are removed, maintaining the integrity of the system. A key improvement here is the enhanced error handling that prompts users to re-enter the account ID if an invalid or non-existent ID is entered. This reduces the risk of accidental deletions and enhances the user experience by providing clear feedback.

```
Admin Operations:
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 8
1. Delete Employee Account
2. Delete Customer Account
Enter your choice: 1
Enter Employee ID to delete: emp1
Employee account with ID emp1 has been deleted.
```

Managing Employee Accounts: In addition to customer accounts, the system also allows for the

management of employee accounts by Admins. This includes adding new employees and deleting existing employee accounts. The system ensures that no two employees have the same ID, maintaining unique identifiers for all system users.

```
Admin Operations:
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 2
Enter new Employee ID: emp1
Employee ID already exists. Please choose a different ID.
```

Search and Filter Capabilities:

Account Search: Admins and Employees can search for accounts based on specific criteria. The system offers a flexible search functionality where users can choose to search by name, date of birth (DOB), or Social Insurance Number (SIN). After selecting the search criterion, users input the relevant information, and the system retrieves and displays all matching accounts. This feature is particularly useful in a banking environment where quick access to account information is critical for customer service and operational efficiency.

```
Admin Operations:
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 6
Enter account number: CHK1
Account Number: CHK1
Customer ID: cust1
Name: First name1 Last name1
Balance: 1000.0
Transactions:
Account opened with initial balance: 1000.0
```

```
Admin Operations:
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 11
Enter Customer ID: cust1
Enter first name: 11
Invalid input: Name must contain only letters. Enter again:
yht
Enter last name: yht
Enter SIN number: 123456789
Report generated successfully.
```

Filtering Accounts: Beyond basic search, the system allows for filtering of account data based on balance or account type. This helps Admins and Employees quickly identify specific groups of accounts, such as those with high balances or accounts that have been inactive for a long period.

Exception Handling and User Feedback:

Input Validation: The system employs comprehensive input validation across all user input fields, including names, phone numbers, emails, SINS, and DOBs. By ensuring that inputs adhere to expected formats, the system minimizes the risk of data entry errors that could lead to system malfunctions or security vulnerabilities. If a user enters invalid data, they are immediately prompted to correct their input, which enhances the overall user experience and data integrity.

```
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 1
1. Open Checking Account
2. Open Savings Account
Enter your choice: 1
Enter first name: 123
Invalid input: Name must contain only letters. Enter again:
yht
Enter last name: yht
Enter password: y@
Invalid input: Password must be at least 5 characters long. Enter again:
y@123
Enter email: yht@gmail.com
Enter phone number: 123456789
Invalid input: Phone number must be exactly 10 digits. Enter again:
1234567890
Enter SIN number: 12
Invalid input: SIN must be exactly 9 digits. Enter again:
123456789
Enter DOB (MM/DD/YYYY):
```

Custom Exceptions: Custom exceptions such as `InvalidAmountException`, `InsufficientFundsException`, and `InputValidationException` are used to handle specific error cases. These exceptions are caught and managed in a way that provides informative error messages to the user, helping them understand what went wrong and how to correct it. This approach prevents abrupt system failures and makes the system more robust and user-friendly.

```
Admin Operations:
1. Open Account
2. Add New Employee
3. Deposit
4. Withdraw
5. Transfer
6. Print Account Details
7. Generate Report
8. Delete Account
9. Generate All Customer Report
10. Show Bank Info
11. Search Account
12. Update Customer Info
13. Logout
Enter your choice: 1
1. Open Checking Account
2. Open Savings Account
Enter your choice: 1
Enter first name: 123
Invalid input: Name must contain only letters. Enter again:
█
```

Detailed Reporting:

Account Reports: The system allows both Admins and Customers to generate detailed reports on account transactions and balances. Admins can generate reports for all customer accounts, while Customers can generate reports for their own accounts. These reports include detailed information such as transaction history, current balance, and personal details. This feature is essential for both internal record-keeping and providing customers with transparency and control over their financial information.

Bank Information Reports: Admins can generate reports that detail information about all employees and other admins in the system. These reports include personal and contact information, job positions, and account activity. This feature is useful for internal audits and ensures that the bank maintains up-to-date records of all personnel.

Thread Safety in Account Transactions:

Concurrent Transaction Handling: The system ensures that multiple users can perform transactions on the same account without data conflicts. This is achieved through the use of thread synchronization techniques, such as the ReentrantLock. By locking critical sections of code where balance updates occur, the system prevents race conditions where multiple threads could potentially modify the account balance simultaneously, leading to inconsistent data. This feature is crucial for maintaining the accuracy and reliability of account data in a multi-user environment.

User-Friendly CLI Design:

Interactive Menus: The system employs interactive command-line menus that guide the user through various operations. These menus are designed to be intuitive, with clear options and prompts that help users navigate the system with ease. The use of ANSI escape codes enhances the user interface by highlighting important information and differentiating between various sections of the menu.

```
*****
*                                     *
*   W E L C O M E   T O   B A N K I N G   *
*           M A N A G E M E N T           *
*           S Y S T E M                   *
*                                     *
*****

Follow the instructions below:

1. Login
2. Exit
Enter your choice: █
```

```
1. Admin Login
2. Employee Login
3. Customer Login
4. Exit
Enter your choice: 1
Enter Admin ID: admin1
Enter Password: adminpass1
Admin login successful.
```

Error Handling and Recovery: The system is designed to handle errors gracefully, ensuring that users are not left with a broken interface or unclear instructions when something goes wrong. For example, when a user enters an invalid input, the system catches the error, provides a helpful error message, and then re-prompts the user to try again. This approach ensures that the user experience remains smooth and frustration-free, even when mistakes are made.

```
*****
*                                     *
*   W E L C O M E   T O   B A N K I N G   *
*           M A N A G E M E N T           *
*           S Y S T E M                   *
*                                     *
*****

Follow the instructions below:

1. Login
2. Exit
Enter your choice: 3
Invalid choice. Please try again.

1. Login
2. Exit
Enter your choice: █
```

Description of Topics Used for Each Feature Developed

Inheritance and Polymorphism

Inheritance and Polymorphism are two of the fundamental principles of object-oriented programming, and they play a crucial role in the design and functionality of the CLI Banking System.

Inheritance:

The system's architecture is designed using inheritance to promote code reuse and maintainability. The Account class is the base class that encapsulates common attributes and methods related to banking accounts, such as account number, customer details, balance, and transaction handling methods. Derived classes like CheckingAccount and SavingsAccount inherit from the Account class, thereby gaining all the properties and methods of the Account class. This setup allows the system to treat different types of accounts uniformly, while still allowing for specialization in the derived classes.

For example, the Account class has methods like deposit(), withdraw(), and transfer(), which are common to all types of accounts. By inheriting from Account, both CheckingAccount and SavingsAccount can use these methods without needing to redefine them, which reduces redundancy and enhances the clarity of the code.

Polymorphism:

Polymorphism in the CLI Banking System is achieved through the use of interfaces and method overriding. The BankOperations interface defines a common set of operations (like deposit, withdraw, transfer) that any type of account must implement. This allows the system to handle different account types polymorphically, meaning that the same code can work with different types of accounts without modification.

For instance, a method that accepts a BankOperations object can operate on any account type (CheckingAccount or SavingsAccount), leveraging the overridden methods in these classes to perform specific actions based on the account type. This flexibility is key in large systems where operations need to be consistent across different object types but also need to cater to the specific behaviors of those objects.

By leveraging inheritance and polymorphism, the system becomes more scalable and easier to extend. New account types can be added with minimal changes to the existing codebase, and common functionality can be centralized in base classes or interfaces, reducing duplication and potential errors.

Generics

Generics are employed in the CLI Banking System to increase the flexibility and type safety of the code. By using generics, the system is able to create data structures and methods that work with any type of object while ensuring that type errors are caught at compile time, rather than at runtime.

Generic Collections:

In the Bank class, collections such as `HashMap<String, Account>`, `HashMap<String, Admin>`, `HashMap<String, Employee>`, and `HashMap<String, Customer>` are used to manage various entities within the system. By parameterizing these collections with specific types (e.g., `Account`, `Admin`, `Employee`), the system ensures that each collection can only store objects of the specified type. This reduces the risk of runtime errors and makes the code easier to understand and maintain.

For example, `HashMap<String, Account>` is used to store accounts with the account number as the key. This setup ensures that only `Account` objects can be stored in this map, and it also allows the system to take advantage of Java's powerful collection framework, including efficient search, insertion, and deletion operations.

Generic Methods:

Generic methods are used to create operations that can handle various data types without needing to overload methods for different types. For instance, the method that searches for accounts by customer information can be written generically to handle different search criteria (name, DOB, SIN) without duplicating code.

This approach not only reduces code duplication but also ensures that the code is more adaptable to future changes. If new search criteria need to be added in the future, the system can be easily extended by leveraging existing generic structures.

Type Safety and Flexibility:

By using generics, the system is able to enforce type safety, meaning that potential type mismatches are caught during compilation rather than at runtime. This reduces the likelihood of bugs related to type conversion or incorrect type usage. It also makes the code more flexible, allowing it to be reused in different contexts without modification.

For example, a generic method to handle transactions might accept a collection of accounts as a parameter, allowing it to process multiple account types in a single operation. The use of generics here allows for code that is both type-safe and flexible, capable of adapting to various use cases with minimal changes.

Generics play a crucial role in making the CLI Banking System both robust and flexible. By abstracting operations over different data types and ensuring type safety, the system becomes easier to extend and maintain, while reducing the risk of runtime errors.

Threading and Concurrency

Threading and Concurrency are vital for ensuring that the CLI Banking System can handle multiple transactions simultaneously without data corruption or inconsistency. This is particularly important in a banking system, where transactions must be processed reliably even under high load.

Concurrency Control with ReentrantLock:

The `ReentrantLock` class is used in the system to manage concurrent access to account data. When multiple threads attempt to modify the same account simultaneously (e.g., through deposits, withdrawals, or transfers), the `ReentrantLock` ensures that only one thread can access the critical section of code at a time. This prevents race conditions and ensures that account balances are updated accurately.

For example, in the `deposit()` and `withdraw()` methods of the `Account` class, `ReentrantLock` is used to lock the account balance while it is being updated. This ensures that even if multiple users try to perform transactions at the same time, the balance is always accurate and reflects all the transactions correctly.

Thread Safety in Transaction Management:

The system's design ensures that all transaction operations are thread-safe. By using locks around critical sections of code where account data is modified, the system prevents issues such as double spending or incorrect balance calculations. This is crucial in a multi-user environment where transactions might be initiated simultaneously by different users.

Thread safety is further enhanced by encapsulating transaction logic within the `Account` class, where all changes to the balance are made. This ensures that all balance-related operations are protected by the same locking mechanism, maintaining consistency across the system.

Scalability and Performance:

By correctly managing concurrency with `ReentrantLock`, the system can scale to handle more transactions without compromising data integrity. This is particularly important in a banking system where transaction volume can vary significantly, and the system must be able to handle peak loads effectively.

The use of threading also improves the performance of the system by allowing multiple operations to be processed in parallel, rather than serially. This reduces waiting times for users and increases the overall throughput of the system.

Threading and concurrency are key aspects of the system that ensure it can handle real-world banking scenarios reliably. By preventing data conflicts and ensuring accurate transaction processing, the system is well-equipped to manage the demands of a multi-user environment.

Exception Handling

Exception Handling is integral to the robustness and reliability of the CLI Banking System. By anticipating and managing errors, the system can provide a better user experience and prevent unexpected crashes or data corruption.

Custom Exceptions:

Custom exceptions are defined to handle specific error conditions within the system. These include `InvalidAmountException` for handling invalid transaction amounts, `InsufficientFundsException` for managing cases where a withdrawal or transfer is attempted with insufficient balance, and `InputValidationException` for catching invalid user inputs during account creation or updates.

For example, if a user tries to withdraw more money than is available in their account, an `InsufficientFundsException` is thrown, and a meaningful error message is displayed to the user. This not only prevents the transaction from proceeding with an invalid state but also informs the user of the exact issue, helping them correct their action.

Input Validation and Error Reporting:

The `Validator` class is used extensively to ensure that all user inputs are valid before they are processed by the system. If an input fails validation (e.g., an incorrectly formatted email or phone number), an `InputValidationException` is thrown, prompting the user to re-enter the correct information. This ensures that only valid data is stored in the system, reducing the risk of errors during processing.

This validation process is critical for maintaining the integrity of the system's data, as it prevents invalid or potentially harmful data from being entered. For instance, ensuring that a SIN number is exactly 9 digits long helps prevent errors in identity verification or report generation.

Graceful Error Handling:

The system is designed to handle errors gracefully, meaning that when an error occurs, the system catches the exception and provides a helpful message to the user. Rather than allowing the system to crash or enter an invalid state, exceptions are caught at critical points, and the user is informed of the issue with clear instructions on how to proceed.

For example, if a user tries to delete an account that doesn't exist, the system catches this scenario and prompts the user to re-enter the account ID. This prevents confusion and ensures that users are guided towards successful completion of their tasks, even when they make mistakes.

Improving User Experience with Exception Handling:

By implementing detailed and specific exception handling, the system enhances the overall user experience. Users are less likely to encounter unhandled errors that could disrupt their workflow, and when errors do occur, they are managed in a way that minimizes frustration and helps users

correct their input.

The system's exception handling also contributes to its reliability, as it ensures that operations are only completed when all conditions are met. This prevents issues such as data corruption, incomplete transactions, or invalid system states, which could otherwise result from unhandled errors.

Exception handling is a critical component of the CLI Banking System's architecture, ensuring that the system is both robust and user-friendly. By managing errors effectively, the system can provide a reliable service, even in the face of unexpected inputs or conditions.